ANÁLISIS NOSQL EN PYTHON:

PRINCIPALES CAUSAS DE RETRASOS EN VUELOS DE EEUU

- Delia Rabanal González
- Master Big data, Data science & business analytics_UCM

1. Introducción

El estudio de grandes volúmenes de datos se ha transformado en un factor fundamental para la toma de decisiones en varios sectores, incluido el transporte aéreo. La creciente demanda de las operaciones aeroportuarias y el gran volumen de vuelos diarios requieren la implementación de tecnologías que puedan almacenar, procesar y analizar datos de manera eficiente.

En este ámbito, las bases de datos NoSQL son una opción muy adecuada frente a los modelos relacionales convencionales.

Mediante nuestro análisis buscamos implementar métodos de análisis de datos mediante una base de datos NoSQL para examinar los retrasos en diferentes vuelos en Estados Unidos. Para ello utilizaremos un extenso database que incluye información mensual sobre vuelos, aerolíneas, aeropuertos y las principales razones de retraso, como el clima, problemas operativos de las aerolíneas, incidencias en el sistema nacional de aviación o llegadas tardías de aviones.

Estos datos han sido extraídos a partir del usuario AbdElaziz El7or, en la plataforma Kaggle.

El significado de cada parámetro lo determinamos a continuación:

- Year: año del registro
- Month: mes del registro
- Carrier: código de la aerolínea
- carrier_name: nombre de la aerolínea
- airport: código del aeropuerto
- airport_name: nombre del aeropuerto
- arr_flights: número total de vuelos que llegaron
- arr_del15: número de vuelos con retraso >= 15 minutos
- carrier_ct: porcentaje de retrasos por problemas de aerolínea
- weather_ct: porcentaje de retrasos por clima
- nas_ct: porcentaje de retrasos por el sistema nacional (NAS)
- security_ct: porcentaje de retrasos por seguridad
- late_aircraft_ct: porcentaje de retrasos por llegada tardía de otra aeronave
- arr_delay: total de minutos de retraso acumulados
- carrier_delay: minutos de retraso causados por la aerolínea
- weather_delay: minutos de retraso por clima
- nas_delay: minutos de retraso por NAS
- security_delay: minutos de retraso por seguridad
- late_aircraft_delay: minutos de retraso por aeronave tardía

2. Descripción de los datos y metodología usada

El análisis que realizaremos muestra diferentes operaciones hechas mediante líneas de código nosql, con el objetivo de mostrar diferentes consultas acerca de las principales causas que llevan a generar retrasos en vuelos. El ámbito geográfico es, por otro lado, Estados Unidos.

Utilizaremos Jupyter Anaconda para realizar dicho análisis. Para ello, primeramente importaremos las librerías necesarias. Éstas son pymongo, el cual contiene comandos nosql, y pandas:

```
!pip install pymongo
from pymongo import MongoClient
import pandas as pd


client = MongoClient("mongodb://localhost:27017/")
db = client["airline_db"]
collection = db["delays"]
```

En este análisis realizaremos operaciones de:

- Importación de datos
- Inserción
- Consulta (filtrado y proyección)
- Actualización
- Agregación (análisis estadístico)

3. Estructura de los datos:

Los documentos utilizados tienen 23 campos en total. Hemos utilizado la mayoría de dichos campos para realizar nuestro estudio, dado que prácticamente casi todos son de valor numérico y nos han servido para filtraciones de datos, hacer comparaciones de los mismos, etc.

Por otro lado, los principales campos que servirán para abarcar las causas principales de los retrasos de las aerolíneas son 5:

- carrier_delay
- weather_delay
- nas_delay
- security_delay
- late_aircraft_delay

4. Inserción, filtrado, proyección y actualización

Primeramente debemos realizar la inserción de los documentos correspondientes. Lo hacemos mediante la siguiente query:

```
df = pd.read_csv(r"C:\Users\Usuario\Desktop\NO_SQL_TAREA\Airline_Delay_Cause.csv")
collection.insert_many(df.head(50).to_dict("records"))
```

Esto agrega un registro de múltiples documentos que albergan la información de vuelos, aerolíneas y sus correspondientes retrasos. Debido a que el csv contiene una gran cantidad de información, es más adecuado representar una pequeña muestra de los primeros datos de la serie para que el programa no colapse:

```
InsertManyResult([ObjectId('696786d336fd83537c5567ff'), ObjectId('696786d336fd83537c556800'), ObjectId('696786d336fd83537c556801'), ObjectId('696786d336f
d83537c556802'), ObjectId('696786d336fd83537c556803'), ObjectId('696786d336fd83537c556804'), ObjectId('696786d336fd83537c556805'), ObjectId('696786d336fd
83537c556806'), ObjectId('696786d336fd83537c556807'), ObjectId('696786d336fd83537c556808'), ObjectId('696786d336fd83537c556809'), ObjectId('696786d336fd8
3537c55680a'), ObjectId('696786d336fd83537c55680b'), ObjectId('696786d336fd83537c55680c'), ObjectId('696786d336fd83537c55680d'), ObjectId('696786d336fd83
537c55680e'), ObjectId('696786d336fd83537c55680f'), ObjectId('696786d336fd83537c556810'), ObjectId('696786d336fd83537c556811'), ObjectId('696786d336fd835
37c556812'), ObjectId('696786d336fd83537c556813'), ObjectId('696786d336fd83537c556814'), ObjectId('696786d336fd83537c556815'), ObjectId('696786d336fd8353
7c556816'), ObjectId('696786d336fd83537c556817'), ObjectId('696786d336fd83537c556818'), ObjectId('696786d336fd83537c556819'), ObjectId('696786d336fd83537
c55681a'), ObjectId('696786d336fd83537c55681b'), ObjectId('696786d336fd83537c55681c'), ObjectId('696786d336fd83537c55681d'), ObjectId('696786d336fd83537c
55681e'), ObjectId('696786d336fd83537c55681f'), ObjectId('696786d336fd83537c556820'), ObjectId('696786d336fd83537c556821'), ObjectId('696786d336fd83537c5
56822'), ObjectId('696786d336fd83537c556823'), ObjectId('696786d336fd83537c556824'), ObjectId('696786d336fd83537c556825'), ObjectId('696786d336fd83537c55
6826'), ObjectId('696786d336fd83537c556827'), ObjectId('696786d336fd83537c556828'), ObjectId('696786d336fd83537c556829'), ObjectId('696786d336fd83537c556
82a'), ObjectId('696786d336fd83537c55682b'), ObjectId('696786d336fd83537c55682c'), ObjectId('696786d336fd83537c55682d'), ObjectId('696786d336fd83537c5568
2e'), ObjectId('696786d336fd83537c55682f'), ObjectId('696786d336fd83537c556830')], acknowledged=True)
```

Por otro lado, usamos el método find_one, el cual permite recuperar un único documento de una colección MongoDB, siendo útil para comprobar la existencia de nuestros datos y verificar la conexión con la base de datos. Por su parte, utilizamos también el método insert_one para insertar un nuevo documento en la colección, añadiendo un registro completo con los campos que hemos definido:

```
doc = collection.find_one()
print(doc)
```

```
{'_id': ObjectId('695ed30b9f97c0d2300f7b2d'), 'year': 2023, 'month': 1, 'carrier': 'XX', 'carrier_name': 'Test Airline', 'airport': 'TEST', 'airport_nam
e': 'Test Airport', 'arr_flights': 100, 'arr_del15': 10, 'arr_delay': 500}
```

```
collection.insert_one({
    "year": 2023,
    "month": 1,
    "carrier": "XX",
    "carrier_name": "Test Airline",
    "airport": "TEST",
    "airport_name": "Test Airport",
    "arr_flights": 100,
    "arr_del15": 10,
    "arr_delay": 500
})
```

```
InsertOneResult(ObjectId('6964bdeb5429828a2542a47c'), acknowledged=True)
```

A continuación, y mediante el filtrado de datos realizamos consultas en Python para obtener información específica. Analizaremos, dentro de este dataset que trata de diferentes tipos de retrasos en el mundo de la aviación, a qué razones se deben dichos retrasos.

Primeramente, analizaremos la causa climática. Para ello además haremos un filtrado de columnas para que sólo aparezcan algunas de ellas, dado que el dataset es de gran amplitud y abarca muchos más datos. El parámetro que mide esta causa climática viene dado por el comando weather_delay, y viene medido en minutos. Por otro lado, observaremos también el parámetro arr_delay, el cual representa el total de minutos acumulados de retraso al llegar a destino.

Haremos varios filtrados, para poder ver diferentes resultados. Primeramente, aeropuertos con retrasos entre 60 y 90 minutos:

```
#RETRASOS A CAUSA DEL CLIMA

for d in collection.find(
    {"weather_delay": {"$gte": 60,
            "$lte": 90},
    "arr_delay": {"$gt": 10}
    },
    {
        "_id": 0,
        "weather_delay": 1,
        "airport" : 1,
        "arr_delay" : 1,
    }
).sort("weather_delay", -1).limit(20):
    print(d)
```

```
{'airport': 'AZO', 'arr_delay': 676.0, 'weather_delay': 90.0}
{'airport': 'BWI', 'arr_delay': 1382.0, 'weather_delay': 90.0}
{'airport': 'LAN', 'arr_delay': 846.0, 'weather_delay': 90.0}
{'airport': 'DLH', 'arr_delay': 364.0, 'weather_delay': 90.0}
{'airport': 'LAX', 'arr_delay': 9062.0, 'weather_delay': 90.0}
{'airport': 'TWF', 'arr_delay': 145.0, 'weather_delay': 90.0}
{'airport': 'BFL', 'arr_delay': 3632.0, 'weather_delay': 90.0}
{'airport': 'BWI', 'arr_delay': 349.0, 'weather_delay': 90.0}
{'airport': 'DLH', 'arr_delay': 364.0, 'weather_delay': 90.0}
{'airport': 'ISP', 'arr_delay': 2449.0, 'weather_delay': 90.0}
{'airport': 'OMA', 'arr_delay': 455.0, 'weather_delay': 90.0}
{'airport': 'PNS', 'arr_delay': 126.0, 'weather_delay': 90.0}
{'airport': 'DTW', 'arr_delay': 434.0, 'weather_delay': 90.0}
{'airport': 'DLH', 'arr_delay': 364.0, 'weather_delay': 90.0}
{'airport': 'LSE', 'arr_delay': 608.0, 'weather_delay': 90.0}
{'airport': 'CHS', 'arr_delay': 2428.0, 'weather_delay': 90.0}
{'airport': 'PNS', 'arr_delay': 1307.0, 'weather_delay': 90.0}
{'airport': 'CHA', 'arr_delay': 1037.0, 'weather_delay': 90.0}
{'airport': 'LAX', 'arr_delay': 2145.0, 'weather_delay': 90.0}
{'airport': 'MLI', 'arr_delay': 838.0, 'weather_delay': 90.0}
```

Representamos solo 20 valores debido a la gran extensión y cantidad de documentos.

Mediante este análisis podemos obsedrvar que el valor máximo de weather_delay (90 minutos) se repite en todos los registros, lo que indica eventos climáticos extremos que impactaron de forma notable las operaciones; por ejemplo, en aeropuertos grandes como LAX y BWI que presentan retrasos totales de llegada muy elevados.

A continuación, haremos estos mismos filtrados para retrasos entre 1h y 1:30h, pero esta vez para otro tipo de causas: por el sistema nacional (NAS), y por causas de seguridad:

```
#RETRASOS POR CAUSAS DE SEGURIDAD

for d in collection.find(
    {"security_delay": {"$gte": 60,
            "$lte": 90},
    "arr_delay": {"$gt": 10}
    },
    {
        "_id": 0,
        "security_delay": 1,
        "airport" : 1,
        "arr_delay" : 1,
    }
).sort("security_delay", -1).limit(20):
    print(d)
```

```
{'airport': 'LAX', 'arr_delay': 2025.0, 'security_delay': 90.0}
{'airport': 'ORD', 'arr_delay': 19545.0, 'security_delay': 90.0}
{'airport': 'SAN', 'arr_delay': 18130.0, 'security_delay': 90.0}
{'airport': 'HNL', 'arr_delay': 9512.0, 'security_delay': 90.0}
{'airport': 'PIE', 'arr_delay': 14126.0, 'security_delay': 90.0}
{'airport': 'MCO', 'arr_delay': 20688.0, 'security_delay': 90.0}
{'airport': 'SAN', 'arr_delay': 28006.0, 'security_delay': 90.0}
{'airport': 'BDL', 'arr_delay': 6817.0, 'security_delay': 90.0}
{'airport': 'DTW', 'arr_delay': 38136.0, 'security_delay': 90.0}
{'airport': 'AVP', 'arr_delay': 479.0, 'security_delay': 90.0}
{'airport': 'MCO', 'arr_delay': 23451.0, 'security_delay': 90.0}
{'airport': 'MIA', 'arr_delay': 3633.0, 'security_delay': 90.0}
{'airport': 'SJC', 'arr_delay': 6871.0, 'security_delay': 90.0}
{'airport': 'ORD', 'arr_delay': 14210.0, 'security_delay': 90.0}
{'airport': 'IND', 'arr_delay': 5616.0, 'security_delay': 90.0}
{'airport': 'BWI', 'arr_delay': 15417.0, 'security_delay': 90.0}
{'airport': 'ABQ', 'arr_delay': 4699.0, 'security_delay': 90.0}
{'airport': 'BNA', 'arr_delay': 27897.0, 'security_delay': 90.0}
{'airport': 'BIL', 'arr_delay': 299.0, 'security_delay': 90.0}
{'airport': 'MIA', 'arr_delay': 9129.0, 'security_delay': 90.0}
```

```python
#RETRASOS A CAUSA DEL NAS

for d in collection.find(
    {"nas_delay": {"$gte": 60,
            "$lte": 90},
    "arr_delay": {"$gt": 10}
    },
    {
        "_id": 0,
        "nas_delay": 1,
        "airport" : 1,
        "arr_delay" : 1,
    }
).sort("nas_delay", 1).limit(20):
    print(d)
```

```
{'airport': 'LAN', 'arr_delay': 448.0, 'nas_delay': 60.0}
{'airport': 'XNA', 'arr_delay': 313.0, 'nas_delay': 60.0}
{'airport': 'TPA', 'arr_delay': 233.0, 'nas_delay': 60.0}
{'airport': 'OKC', 'arr_delay': 324.0, 'nas_delay': 60.0}
{'airport': 'EUG', 'arr_delay': 808.0, 'nas_delay': 60.0}
{'airport': 'MDW', 'arr_delay': 1295.0, 'nas_delay': 60.0}
{'airport': 'GCK', 'arr_delay': 139.0, 'nas_delay': 60.0}
{'airport': 'BNA', 'arr_delay': 411.0, 'nas_delay': 60.0}
{'airport': 'LAN', 'arr_delay': 375.0, 'nas_delay': 60.0}
{'airport': 'SAT', 'arr_delay': 595.0, 'nas_delay': 60.0}
{'airport': 'BNA', 'arr_delay': 160.0, 'nas_delay': 60.0}
{'airport': 'LAN', 'arr_delay': 280.0, 'nas_delay': 60.0}
{'airport': 'CSG', 'arr_delay': 481.0, 'nas_delay': 60.0}
{'airport': 'MQT', 'arr_delay': 412.0, 'nas_delay': 60.0}
{'airport': 'DAL', 'arr_delay': 407.0, 'nas_delay': 60.0}
{'airport': 'ELP', 'arr_delay': 191.0, 'nas_delay': 60.0}
{'airport': 'MTJ', 'arr_delay': 786.0, 'nas_delay': 60.0}
{'airport': 'DTW', 'arr_delay': 395.0, 'nas_delay': 60.0}
{'airport': 'ANC', 'arr_delay': 676.0, 'nas_delay': 60.0}
{'airport': 'DAL', 'arr_delay': 60.0, 'nas_delay': 60.0}
```

Tras haber analizado estos campos, pasaremos a ejecutar funciones de actualización. Realizaremos esto mediante comandos update_many; a través de los cuales vamos a añadir nuevas etiquetas de clasificación. A continuación mostramos la query correspondiente:

```python
result = collection.update_many(
    {
        "year": 2019,
        "month": 9,
        "arr_flights": {"$gt": 100}
    },
    [
        {
            "$set": {
                "busy_airport": True,
                "delay_percentage": {
                    "$cond": [
                        {"$gt": ["$arr_flights", 0]},
                        {
                            "$multiply": [
                                {"$divide": ["$arr_del15", "$arr_flights"]},
                                100
                            ]
                        },
                        0
                    ]
                },
                "updated_at": datetime.utcnow()
            }
        }
    ]
)

print("Matched:", result.matched_count)
print("Modified:", result.modified_count)
```

```
Matched: 10176
Modified: 10176
```

Como podemos observar, hemos realizado una actualización bastante destacada de los documentos que corresponden a septiembre del año 2019 (seleccionando previamente los aeropuertos con más de 100 vuelos de llegada). A los 10176 documentos que se han encontrado le hemos añadido el comando busy_airport cono True, para indicar que se trata de aeropuertos bastante concluidos.

Por otro lado, hemos también añadido el porcentaje de vuelos con bastante retraso a través de la etiqueta delay_percentage. Para ello hemos dividido los vuelos con más de 15 minutos de retraso respecto al total de las llegadas.

Al comienzo de nuestro análisis, hemos añadido las librerías correspondientes para poder trabajar con los datos. Añadimos también la librería datetime precisamente para este apartado del trabajo; con ella, podemos registrar el momento exacto en el que se hacen estas actualizaciones.

El resultado confirma que los documentos coincidentes fueron modificados correctamente, lo que valida que la operación se ejecutó según lo previsto.

Como analizamos en el primer punto de este análisis, existen otras principales razones también que determinan los retrasos en los diferentes aeropuertos del país, como comentamos anteriormente; por ejemplo, problemas con la propia aeronave (llegada tardía, problemas técnicos de estructura, repostaje de combustible, etc).

En el siguiente apartado mostraremos esto con un ejemplo de agregación con comandos nosql. Para ejemplificarlo calcularemos el total de retrasos por aeronave mostrados en la base de datos, lo cual se calcula directamente con la suma de la columna late_aircraft_delay:

```python
#TOTAL DE RETRASOS POR AERONAVE

pipeline = [
    {"$group": {
        "_id": "$carrier_name",
        "total_delay": {"$sum": "$late_aircraft_delay"}
    }},
    {"$sort": {"total_delay": -1}},
    {"$limit": 6}
]

for d in collection.aggregate(pipeline):
    print(d)
```

```
{'_id': 'Southwest Airlines Co.', 'total_delay': 1269896904.0}
{'_id': 'Spirit Air Lines', 'total_delay': 55649028.0}
{'_id': 'Atlantic Coast Airlines', 'total_delay': 24011952.0}
{'_id': 'Hawaiian Airlines Inc.', 'total_delay': 16064172.0}
{'_id': 'Horizon Air', 'total_delay': 4866564.0}
{'_id': 'Aloha Airlines Inc.', 'total_delay': 1168776.0}
```

Analizaremos también dos parámetros más: retrasos medios por aeropuerto (el cual representaremos como avg_delay), y a continuación crearemos un top 10 de los aeropuertos con más retrasos en Estados Unidos:

```
#RETRASO MEDIO POR AEROPUERTO
pipeline = [
    {
        "$group": {
            "_id": "$airport",
            "avg_delay": {"$avg": "$arr_delay"}
        }
    },
    {"$sort": {"avg_delay": -1}},
    {"$limit": 10}
]

for d in collection.aggregate(pipeline):
    print(d)
```

```
{'_id': 'ATL', 'avg_delay': 4642.7501491434905}
{'_id': 'DFW', 'avg_delay': 3427.254462097402}
{'_id': 'SFO', 'avg_delay': 2689.9436038090544}
{'_id': 'SFB', 'avg_delay': 2673.5754716981132}
{'_id': 'IAH', 'avg_delay': 2229.191135734072}
{'_id': 'DAL', 'avg_delay': 2097.4776394651913}
{'_id': 'MCO', 'avg_delay': 1653.0125345095676}
{'_id': 'LAS', 'avg_delay': 1568.048129389708}
{'_id': 'PGD', 'avg_delay': 1427.4416666666666}
{'_id': 'DTW', 'avg_delay': 1425.670057057802}
```

A través de esta query observamos que el retraso medio de llegada (avg_delay) por aeropuerto, ordenados de mayor a menor.

Observamos que ATL (Atlanta) presenta el mayor retraso medio, seguido por DFW (Dallas Fort Worth) y SFO (San Francisco). Esto sugiere que los aeropuertos con mayor volumen de tráfico tienden a acumular retrasos más elevados. Factores como la congestión, condiciones meteorológicas adversas y la alta rotación de vuelos pueden influir en estos valores.

Además, algunos aeropuertos turísticos como MCO (Orlando) o LAS (Las Vegas) también aparecen en el ranking, lo que puede estar relacionado con picos de demanda estacional.

```python
#LOS 10 AEROPUERTOS CON MAS RETRASOS
pipeline = [
    {"$group": {
        "_id": "$airport_name",
        "total_delay": {"$sum": "$arr_delay"}
    }},
    {"$sort": {"total_delay": -1}},
    {"$limit": 10}
]

for d in collection.aggregate(pipeline):
    print(d)
```

```
{'_id': 'Atlanta, GA: Hartsfield-Jackson Atlanta International', 'total_delay': 163429448.0}
{'_id': 'Dallas/Fort Worth, TX: Dallas/Fort Worth International', 'total_delay': 118860612.0}
{'_id': 'San Francisco, CA: San Francisco International', 'total_delay': 87285980.0}
{'_id': 'Houston, TX: George Bush Intercontinental/Houston', 'total_delay': 68402730.0}
{'_id': 'Las Vegas, NV: McCarran International', 'total_delay': 56037336.0}
{'_id': 'Orlando, FL: Orlando International', 'total_delay': 52091384.0}
{'_id': 'Detroit, MI: Detroit Metro Wayne County', 'total_delay': 51721884.0}
{'_id': 'Seattle, WA: Seattle/Tacoma International', 'total_delay': 41075912.0}
{'_id': 'San Diego, CA: San Diego International', 'total_delay': 27020736.0}
{'_id': 'Raleigh/Durham, NC: Raleigh-Durham International', 'total_delay': 21292188.0}
```

Por último, estos resultados muestran los 10 aeropuertos con mayor retraso total acumulado, es decir, la suma de todos los minutos de retraso de llegada registrados.

El aeropuerto con mayor retraso total es Hartsfield-Jackson Atlanta International, seguido por Dallas/Fort Worth International y San Francisco International. Esto se debe a que son los principales aeropuestos de mayor volumen en todo el país. Por ello las cifras son iguales al anterior análisis.

5. Conclusiones

Tras los análisis efectuados, podemos indicar que los retrasos en los vuelos no están distribuidos de forma equitativa entre aeropuertos y compañías aéreas. Por ejemplo, los aeropuertos más grandes que funcionan como puntos centrales, como Atlanta, Dallas Fort Worth o San Francisco, agrupan tanto los mayores retrasos promedio como el volumen total más alto de retrasos, lo cual está directamente vinculado con su alto tráfico aéreo y complejidad en las operaciones. Además, algunas compañías concentran un volumen significativamente mayor de retrasos, lo que sugiere posibles problemas operativos.

Llevando estas conclusiones a un ámbito más técnico dentro del análisis de datos, observamos claramente que la utilización de los comandos NoSql y los pipelines de agregación han facilitado llevar a cabo cálculos complejos de manera efectiva y adaptable.

En resumen, nuestro estudio valida que las bases de datos NoSql son una herramienta muy eficaz para el análisis de datos en entornos reales, como en la industria

aeronáutica, donde el volumen, la diversidad y la rapidez de los datos precisan de soluciones flexibles.